Bu yapıda **`bindings`**, UMFS çekirdeğinin farklı programlama dillerinden kullanılabilmesini sağlayan **arayüz katmanı** anlamına geliyor.

Örnekte:

bindings/
├── python
├── rust
├── go
└── javascript

fikri şu:

Python Uygulaması
    |
    ▼
Python Binding
    |
    ▼
UMFS Core (C++)

Burada asıl iş mantığı ve performans kritik kodlar **C++ ile yazılmıştır**. Python binding ise C++ fonksiyonlarını Python'dan çağırabilmeni sağlar.

Örneğin C++ tarafında:

bool create_volume(const std::string& name);

fonksiyonu varsa, Python binding sayesinde:

import umfs

umfs.create_volume("cluster01")

şeklinde kullanılabilir.

Aynı mantık Rust için:

use umfs;

umfs::create_volume("cluster01");

Go için:

umfs.CreateVolume("cluster01")

olabilir.

Bu yaklaşımın avantajı:

- Tek çekirdek kodu (C++)
- Maksimum performans
- Birden fazla dil desteği
- Python'a bağımlı olmama
- PyTorch, TensorFlow, OpenCV ve Qt'nin kullandığı mimariye benzer yapı

Yani metindeki "**Python sadece binding**" ifadesi:

"UMFS'nin asıl motoru Python ile yazılmasın. Çekirdek C++ olsun. Python sadece o çekirdeğe erişim sağlayan bir dış arayüz olsun."

anlamına geliyor. Bu, özellikle senin sistem programlama ve yüksek performans hedeflerin açısından oldukça yaygın ve mantıklı bir mimaridir.